

[IS216] Extra Exercises - Vue Basics I

Objectives

- Gain a solid understanding of reactive programming concepts.
- Learn to leverage Vue framework features for building efficient web applications.
- Practice applying Vue directives to create dynamic and responsive web pages.

Instructions

- Questions without an asterisk are easy.
- Questions marked with * are moderately challenging.
- Questions marked with ** are challenging.
- Questions marked with *** are very challenging.

Contents

Question 1 - Puzzle (*)	2
Question 2 - Maximum Characters (*/**)	3
Question 3 - Form Validation (*)	5
Question 4 - Choose Background Color (*)	8
Question 5 - IS Curriculum (*)	9
Question 6 - Stars (**)	11
Question 7 - Currency Converter (*)	13
Question 8 - Measurement Converter (**/**)	15

Note:

- If you notice any mistakes or errors in the questions, please email your instructors with a description of the issue. We will address it as soon as possible.
- For the best viewing experience, please use Microsoft Office 365 Online.

Question 1 - Puzzle (*)

Background:

You are tasked with developing a simple click-to-reveal web application. The page should display a series of sentences, with one of the sentences having part of its text hidden behind a “?” button (see Figure q1a). When the button is clicked, the hidden text is revealed (see Figure q1b).

Requirements:

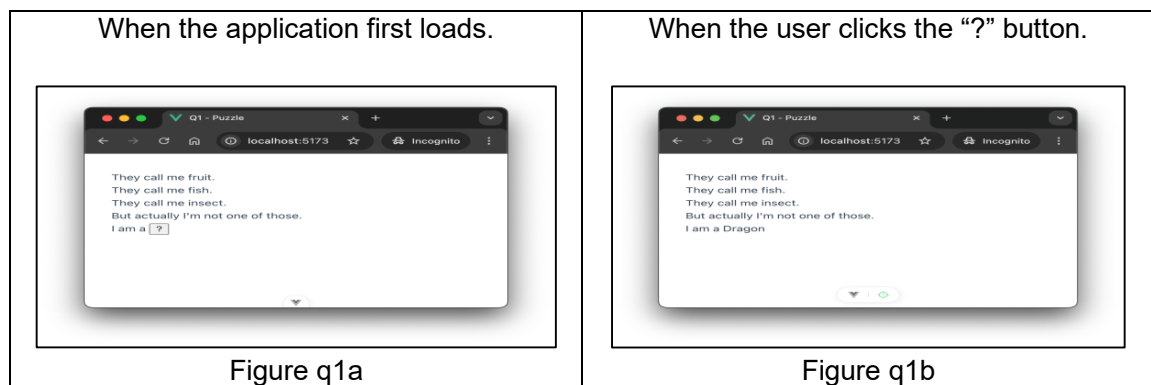
- When the application first loads (see Figure q1a):
 - The web page should display five sentences, each starting on a new line.
 - The last sentence must end with a “?” button.
- When the user clicks the “?” button (see Figure q1b):
 - The button should be replaced with the word “Dragon”.
 - The reveal should be a one-time action (no toggle required).

Instructions:

1. Navigate to the q1 directory in your terminal.
2. Run the command “pnpm install” to install the required dependencies.
3. Run the command “pnpm dev” to run the app in your development server.
4. After installation, navigate to the q1/src directory.
5. Open the App.vue file and add or modify the code to fulfil the requirements described above.

Notes:

- The recommended browser width for viewing all the figures is 500px.



Question 2 - Maximum Characters (*/**)

Background:

You are tasked with developing a text input component for a web messaging application. The component should include a character counter and a text input area with a predefined character limit (see Figure q2a). As the user approaches the character limit, the background color of the text input area should change to visually warn the user (see Figure q2b). Once the maximum character limit is reached, the input should prevent any additional characters from being entered or displayed (see Figure q2c).

Requirements:

Part A (*)

- When the application first loads (see Figure q2a):
 - The text input component should be pre-filled with the text: “What if I told you Vue can do that”.
 - The character counter should display the number of characters of this predefined text.
- As the user continues typing:
 - The character counter should increase to reflect the total number of characters currently entered in the text input component. This can be done by binding the text property in the Vue instance to the text input using two-way data binding.

Part B (**)

- When 40 characters have been entered (see Figure q2b):
 - The background color of the text input area should change by applying the provided CSS class warn. This can be done by conditionally binding the warn class to the <textarea> element when the number of entered characters is 40 or more.
- When the input reaches 50 characters (see Figure q2c):
 - The background color should remain unchanged. At this point, no additional characters should be accepted or displayed in the text input. This can be implemented by binding the maxlength attribute to the limit data property.
 - The character counter should display 50, reflecting the maximum character limit.

Instructions:

1. Navigate to the q2 directory in your terminal.
2. Run the command “pnpm install” to install the required dependencies.
3. Run the command “pnpm dev” to run the app in your development server.
4. After installation, navigate to the q2/src directory.
5. Open the App.vue file and add or modify the code to fulfil the requirements described above.

Notes:

- The recommended browser width for viewing all the figures is 500px.

When the application first loads.

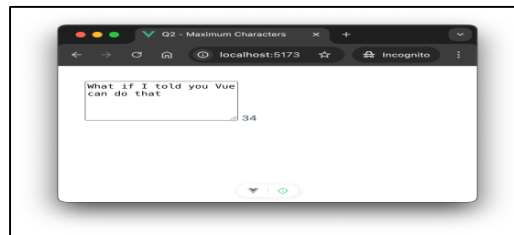


Figure q2a

When 40 characters have been entered.

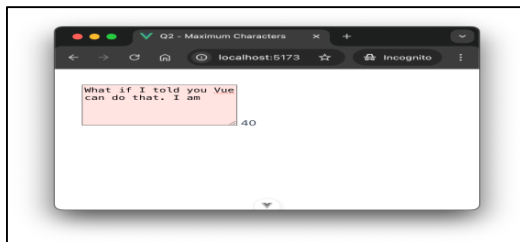


Figure q2b

When the input reaches 50 characters.

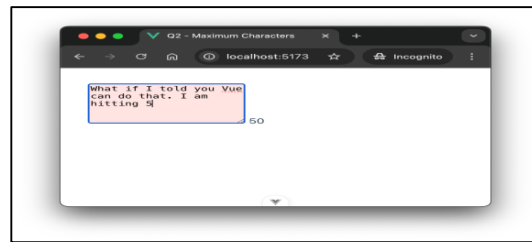


Figure q2c

Question 3 - Form Validation (*)

Background:

You are tasked with implementing form input validation for a web application's password reset page. The form includes multiple text input fields and a "Submit" button (see Figure q3a). The password reset will only be successful if all user inputs satisfy the specified validation criteria. If any input is invalid, a warning message will be displayed to guide the user in correcting the input (see Figures q3b to q3e).

Requirements:

- When the application first loads (see Figure q3a):
 - The password reset page should display four text input fields: Username, Current Password, New Password, and Confirm New Password, along with a "Submit" button.
 - The warning message should be hidden by default. This can be done by using `v-if` to conditionally display the message only when needed.
- When the form is submitted:
 - If any required input field is left empty (see Figure q3b):
 - Display the warning message: "All inputs must be filled!"
 - If the "New Password" and "Confirm New Password" fields do not match (see Figure q3c):
 - Display the warning message: "New passwords do not match!"
 - If the entered username does not exist in the predefined user's array in the Vue instance (see Figure q3d):
 - Display the warning message: "Username does not exist!"
 - If the entered current password does not match the stored password associated with the given username (see Figure q3e):
 - Display the warning message: "Current password incorrect!"

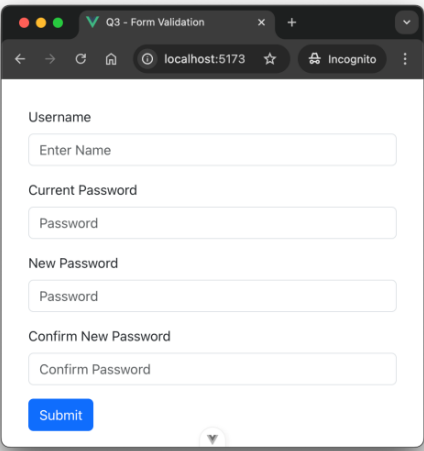
Instructions:

1. Navigate to the q3 directory in your terminal.
2. Run the command `pnpm install` to install the required dependencies.
3. Run the command `pnpm dev` to run the app in your development server.
4. After installation, navigate to the q3/src directory.
5. Open the App.vue file and add or modify the code to fulfil the requirements described above.

Notes:

- The recommended browser width for viewing all the figures is 500px.
- Use Bootstrap's alert component with the code below to display the warning messages:
 - `<div class="alert alert-warning" role="alert">Input your warning message here.</div>`

When the application first loads.

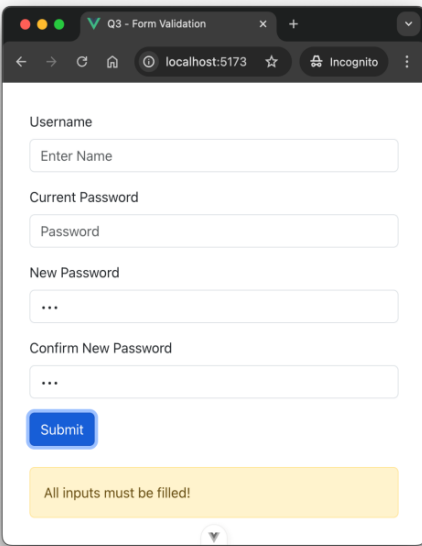


A screenshot of a web browser window titled "Q3 - Form Validation" showing a form with four input fields: "Username" (placeholder "Enter Name"), "Current Password" (placeholder "Password"), "New Password" (placeholder "Password"), and "Confirm New Password" (placeholder "Confirm Password"). A blue "Submit" button is at the bottom. The browser address bar shows "localhost:5173" and "Incognito".

Figure q3a

When the following details are entered:

- Username: (left blank)
- Current Password: (left blank)
- New Password: 123
- Confirm New Password: 123

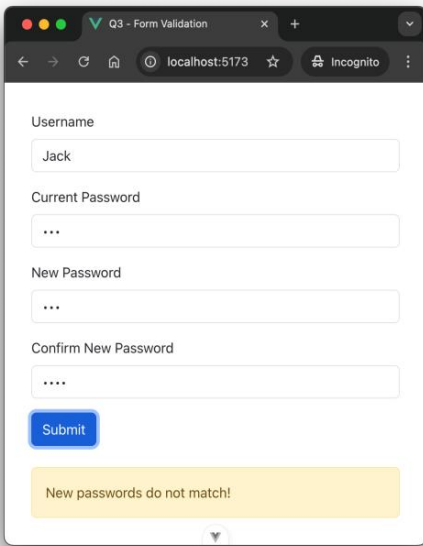


A screenshot of the web browser window showing the form with the same inputs as Figure q3a. A yellow error message box at the bottom states "All inputs must be filled!". The "Submit" button is still visible.

Figure q3b

When the following details are entered:

- Username: Jack
- Current Password: abc
- New Password: 123
- Confirm New Password: 1234

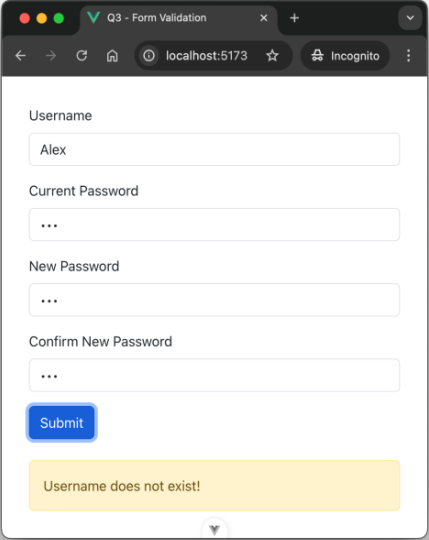


A screenshot of the web browser window showing the form with the inputs "Jack", "abc", "123", and "1234". A yellow error message box at the bottom states "New passwords do not match!". The "Submit" button is still visible.

Figure q3c

When the following details are entered:

- Username: Alex
- Current Password: abc
- New Password: 123
- Confirm New Password: 123

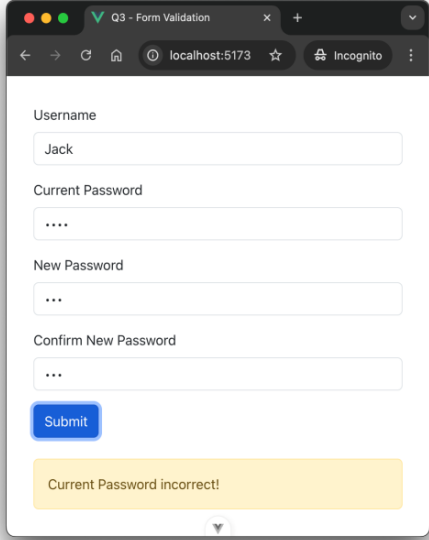


A screenshot of a web browser window displaying a form titled "Q3 - Form Validation". The browser's address bar shows "localhost:5173" and "Incognito". The form contains four input fields: "Username" (containing "Alex"), "Current Password" (containing "..."), "New Password" (containing "..."), and "Confirm New Password" (containing "..."). Below the fields is a blue "Submit" button. At the bottom of the form, a yellow error message box displays the text "Username does not exist!".

Figure q3d

When the following details are entered:

- Username: Jack
- Current Password: abcd
- New Password: 123
- Confirm New Password: 123



A screenshot of a web browser window displaying a form titled "Q3 - Form Validation". The browser's address bar shows "localhost:5173" and "Incognito". The form contains four input fields: "Username" (containing "Jack"), "Current Password" (containing "..."), "New Password" (containing "..."), and "Confirm New Password" (containing "..."). Below the fields is a blue "Submit" button. At the bottom of the form, a yellow error message box displays the text "Current Password incorrect!".

Figure q3e

Question 4 - Choose Background Color (*)

Background:

You are tasked with developing a color picker application. The application should allow users to dynamically change the background color behind a kitten image (see Figures q4a and q4b).

Requirements:

- When the application first loads (see Figure q4a):
 - It should display a color picker with black as the default selected color, and the kitten image should appear with a black background.
- When a new color is selected (see Figure q4b):
 - The background color behind the image should update instantly and reactively, without requiring a page reload.

Instructions:

1. Navigate to the q4 directory in your terminal.
2. Run the command “pnpm install” to install the required dependencies.
3. Run the command “pnpm dev” to run the app in your development server.
4. After installation, navigate to the q4/src directory.
5. Open the App.vue file and add or modify the code to fulfil the requirements described above.

Notes:

- The recommended browser width for viewing all the figures is 500px.
- The image file kitten.png is provided in the public/img folder.
- You may use your own CSS or Bootstrap to style the page. Consider using these Bootstrap utility classes to format the layout and appearance:
 - m-* for margin
 - p-* for padding
 - img-fluid to make the image responsive

When the application first loads.

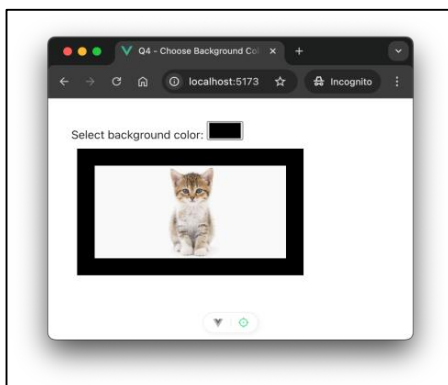


Figure q4a

When the pinkish-red color is selected.

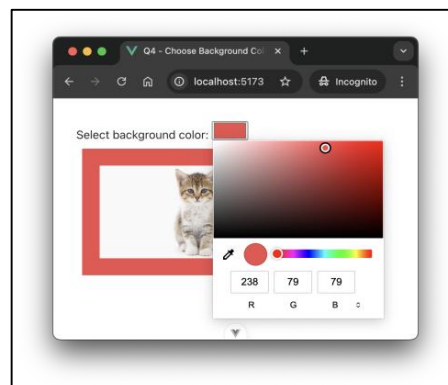


Figure q4b

Question 5 - IS Curriculum (*)

Background:

You are tasked with developing a web application that displays the courses offered under the Information Systems major based on selected course categories. The application should allow users to choose a course category, and the corresponding list of courses should be automatically displayed on the page (see Figures q5a and q5b).

Requirements:

- When the application first loads (see Figure q5a):
 - A drop-down list should be displayed, containing a series of course categories. These category names should be dynamically extracted from the `is_curriculum` data. This can be done by defining a computed property that returns an array of unique course categories, such as ["University Core", "IS Major Core", ...]. Use the `v-for` directive within the `<option>` element to loop through this array and populate the drop-down menu.
 - By default, the category "University Core" should be selected, and its corresponding list of courses should be displayed. This can be implemented by defining another computed property that returns the array of courses corresponding to the selected category, such as ["Statistics", "Computational Thinking", ...]. Use the `v-for` directive within the `<li>` element to iterate through and display each course.
- When a different category is selected (see Figure q5b):
 - The course list should automatically update to reflect the courses associated with the newly selected category. This can be done by binding the `selected_cat` data property in the Vue instance to the `<select>` input using two-way data binding with the `v-model` directive.

Instructions:

1. Navigate to the `q5` directory in your terminal.
2. Run the command `"pnpm install"` to install the required dependencies.
3. Run the command `"pnpm dev"` to run the app in your development server.
4. After installation, navigate to the `q5/src` directory.
5. Open the `App.vue` file and add or modify the code to fulfil the requirements described above.

Notes:

- The recommended browser width for viewing all the figures is 500px.
- The data for course categories and their corresponding courses is provided in the Vue instance's data property, under `is_curriculum`.

When the application first loads.

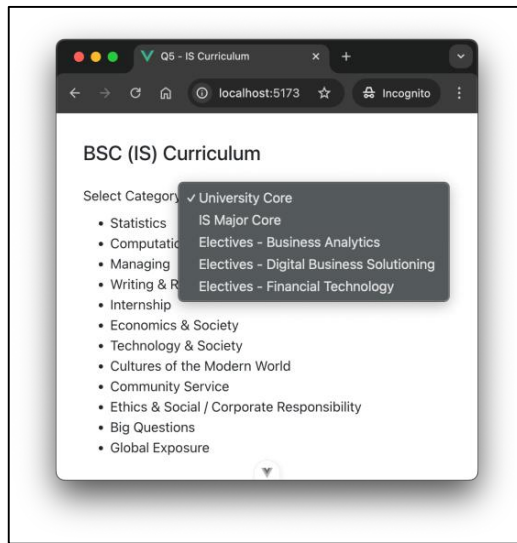


Figure q5a

When "IS Major Core" category is selected.

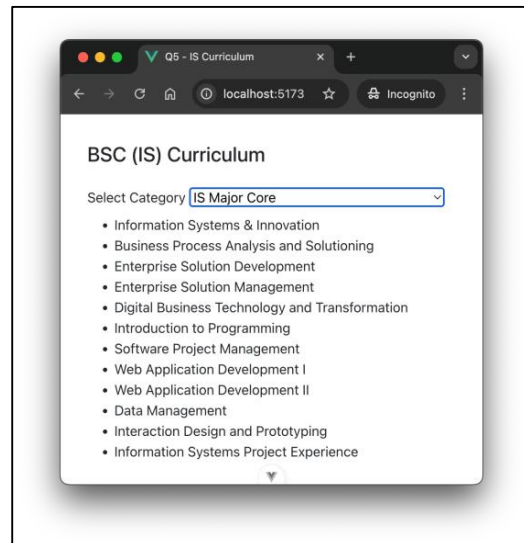


Figure q5b

Question 6 - Stars (**)

Background:

You are tasked with developing a symbol display web application. The application should generate and display two different types of symbol patterns (either asterisks ('*') or dollar signs ('\$')) based on the symbol type and number specified by the user (see Figure q6b and q6c).

Requirements:

- When the application first loads (see Figure q6a):
 - It should display a text input field and two radio buttons for the user to enter a number and select a symbol.
 - The text input should default to 0, and the "*" symbol should be selected by default.
- When a number is entered with the default "*" symbol selected (see Figure q6b):
 - Pattern One should display a single row of "*" repeated based on the number entered. This can be done by creating a computed property that constructs a string of the selected symbol repeated the specified number of times.
 - Pattern Two should display multiple rows, where each row contains an increasing number of "*" from 1 up to the number entered. This can be done by creating a computed property that constructs an HTML string with line breaks using the `<br>` element to display the pattern across multiple lines.
- When the symbol is changed to "\$" (see Figure q6c):
 - Both Pattern One and Pattern Two should update automatically to use the "\$" symbol instead of "*". The overall structure and logic remain the same, only the symbol changes. This can be done by binding the selected data property in the Vue instance to the radio button input using two-way data binding with the `v-model` directive.

Instructions:

1. Navigate to the q6 directory in your terminal.
2. Run the command `pnpm install` to install the required dependencies.
3. Run the command `pnpm dev` to run the app in your development server.
4. After installation, navigate to the q6/src directory.
5. Open the `App.vue` file and add or modify the code to fulfil the requirements described above.

Notes:

- The recommended browser width for viewing all the figures is 500px.

When the application first loads.

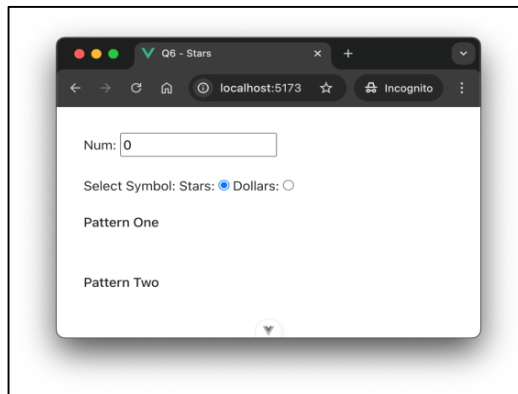


Figure q6a

When the number 3 is entered with the default “*” symbol selected.

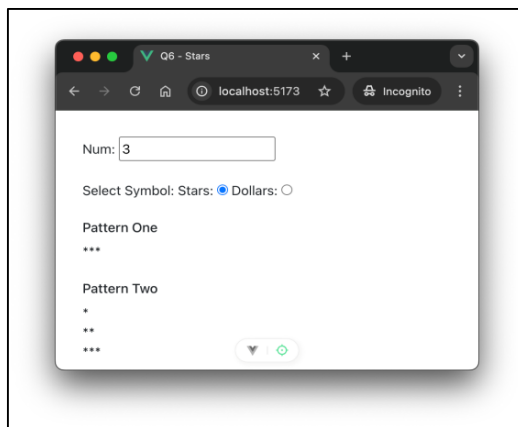


Figure q6b

When the number remains at 3 and the symbol is changed to "\$".

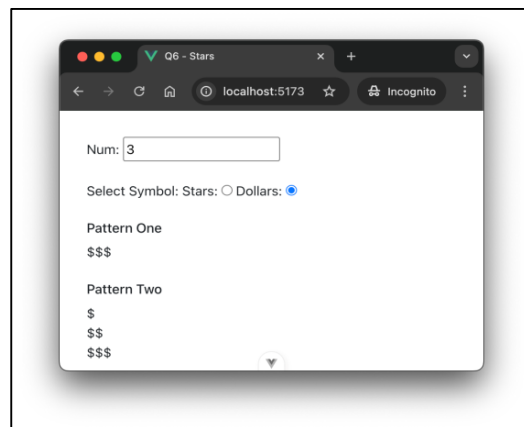


Figure q6c

Question 7 - Currency Converter (*)

Background:

You are tasked with developing a currency converter web application. The application should convert a specified amount in Singapore dollars (SGD) to its equivalent value in a user-selected foreign currency (see Figure q7a).

Requirements:

- When the application first loads (see Figure q7a):
 - It should display a text input field, a drop-down menu and the converted amount in the default-selected foreign currency display. The text input should default to 1, and “United States Dollar” should be pre-selected in the drop-down list.
 - The list of currencies in the drop-down menu should be dynamically generated from the `exchange_rates` object in the Vue instance's data property. This can be done by creating a computed property that returns an array of currency names extracted from the object's keys.
 - The converted amount display can be done using a computed property that performs the calculation based on the relevant data whenever the user inputs change.
- When either the text input or the selected option in the drop-down menu is changed (see Figure q7b):
 - The conversion amount should update automatically and immediately to reflect the changes. This can be done by binding the `sing_dollar` and `selected_currency` data property in the Vue instance to the text input field and drop-down menu respectively, using two-way data binding with the `v-model` directive.

Instructions:

1. Navigate to the `q7` directory in your terminal.
2. Run the command `pnpm install` to install the required dependencies.
3. Run the command `pnpm dev` to run the app in your development server.
4. After installation, navigate to the `q7/src` directory.
5. Open the `App.vue` file and add or modify the code to fulfil the requirements described above.

Notes:

- The recommended browser width for viewing all the figures is 500px.
- The converted exchange rate results should be displayed with 6 decimal places.
- Consider using `Object.keys()` to access currency options and `toFixed(6)` to format the output.

When the application first loads.

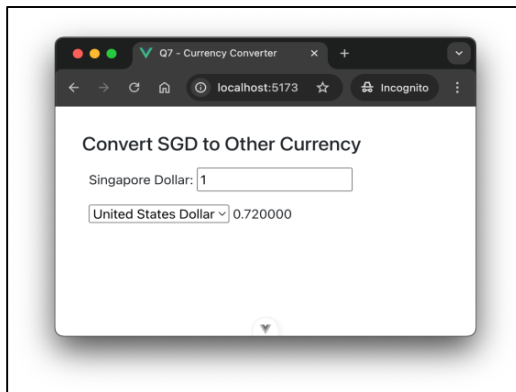


Figure q7a

When the text input is changed to "5" and "Malaysia Ringgit" option is selected.

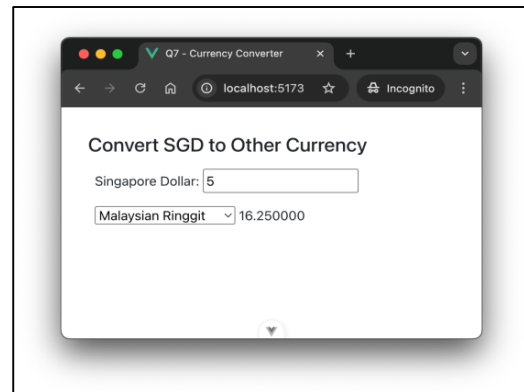


Figure q7b

Question 8 - Measurement Converter (**/**)

Background:

You are tasked with developing a measurement converter web application. The application should convert a user-provided value from one unit of measurement to another, based on the selected conversion type.

Requirements:

Part A (**)

- When the application first loads (see Figure q8a to q8d):
 - It should display
 - A text input field for the user to enter the value to convert (see Figure q8a).
 - A disabled text input field to show the conversion result (see Figure q8b).
 - A drop-down menu to select the measurement type (e.g., Length, Volume) (see Figure q8c).
 - Two drop-down menus to choose the units for conversion (see Figure q8d).
 - The options in the measurement type drop-down menu should be dynamically generated from the keys of the measurement object defined in the Vue instance's data property. For example: "Length", "Volume", "Digital", and "Mass". This can be done by creating a computed property that returns an array of measurement type extracted from the object's keys.
 - The options in the unit's drop-down menu should be dynamically generated from the keys of the selected measurement type object within the measurement data property in the Vue instance. For example, if the measurement type "Length" is selected, the unit's drop-down menu should display only "Inch", "centimetre", "Yard", and "Mile". This can be done by creating a computed property that returns an array of unit conversion keys (e.g., "Inch2centimetre") extracted from the selected measurement type object. These keys can then be processed to extract and return only the distinct units.

Part B (***)

- When the user makes changes to the text input field or select different measurement units ((See Figure q8e to q8j):
 - The conversion result should update automatically and immediately to reflect the changes. This can be done by binding the respective data properties in the Vue instance to the corresponding user input fields using two-way data binding with the v-model directive.

Instructions:

1. Navigate to the q8 directory in your terminal.
2. Run the command "pnpm install" to install the required dependencies.
3. Run the command "pnpm dev" to run the app in your development server.
4. After installation, navigate to the q8/src directory.
5. Open the App.vue file and add or modify the code to fulfil the requirements described above.

Notes:

- The recommended browser width for viewing all the figures is 500px.
- Explore the use of `Object.keys()`, `split()`, and `includes()` for working with objects and strings.
- You are provided with a set of data to be used for converting values from one unit of measurement to another. For example, under the 'Length' category, the conversion from inches to centimeters is given as 2.54, meaning 1 inch is equivalent to 2.54 centimeters. In the 'Volume' category, the conversion from gallons to litres is 4.54609, meaning 1 gallon is equivalent to 4.54609 litres.

Text input field.

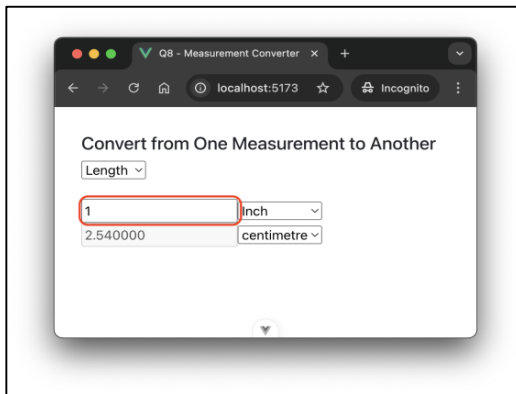


Figure q8a

Disabled text input field.

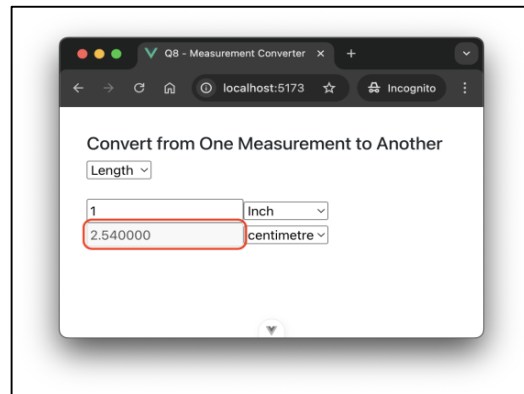


Figure q8b

Measurement Types

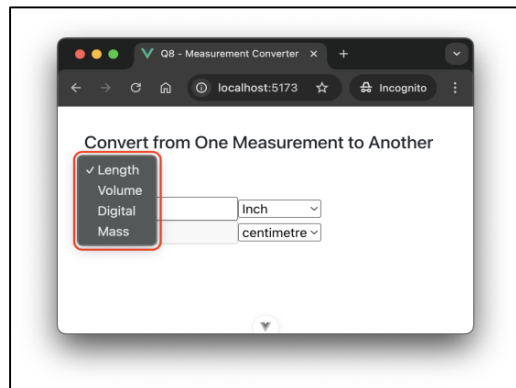


Figure q8c

Conversion Units

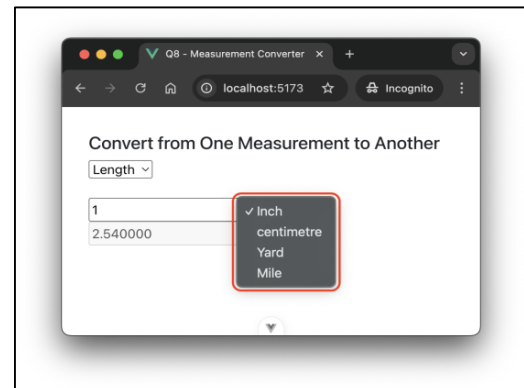


Figure q8d

When the user enters the value '2', selects 'Inch' as the source unit and 'Centimetre' as the target unit.

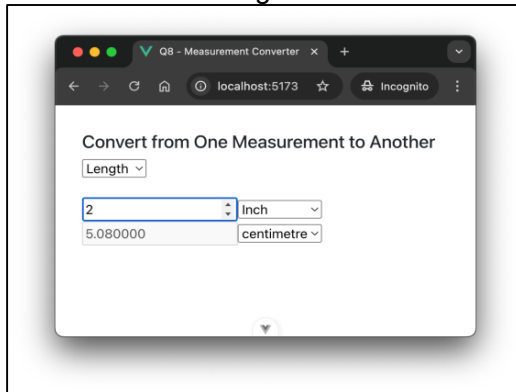


Figure q8e

When the user changes the target unit to 'Yard' while keeping the rest unchanged.

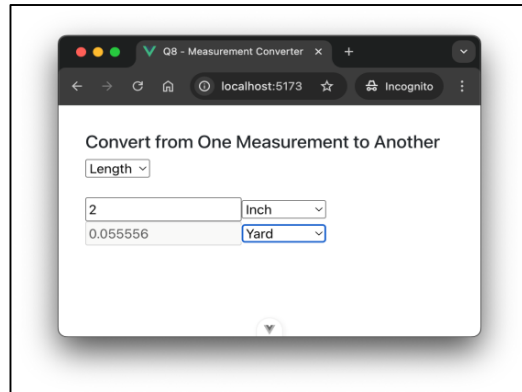


Figure q8f

When the user changes the measurement type to 'Volume' and selects 'litre' as the source unit and 'gallon' as the target unit.

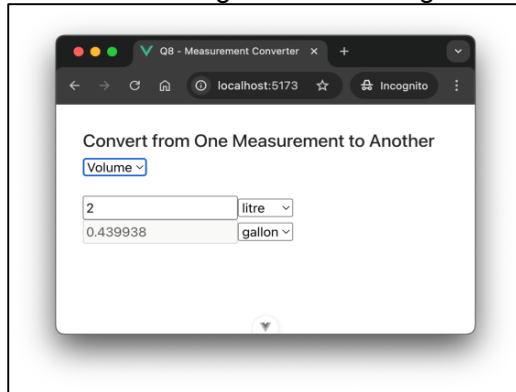


Figure q8g

When the user swaps the units by selecting 'gallon' as the source unit and 'litre' as the target unit.

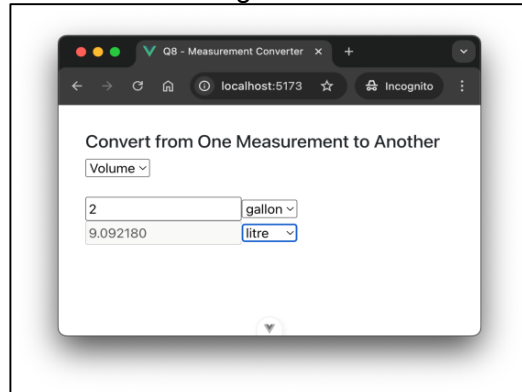


Figure q8h

When the user changes the measurement type to 'Mass' and selects 'Ounce' as the source unit and 'Gram' as the target unit.

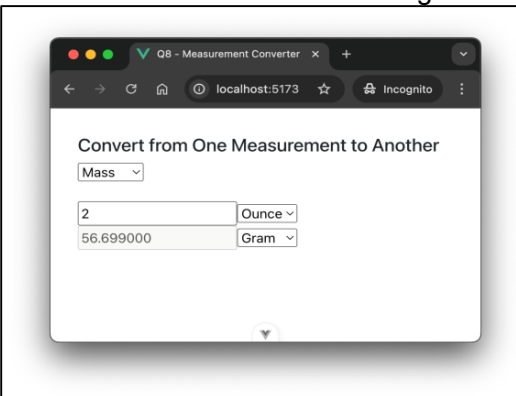


Figure q8i

When the user enters the value '3', selects 'Ounce' as the source unit and 'Pound' as the target unit.

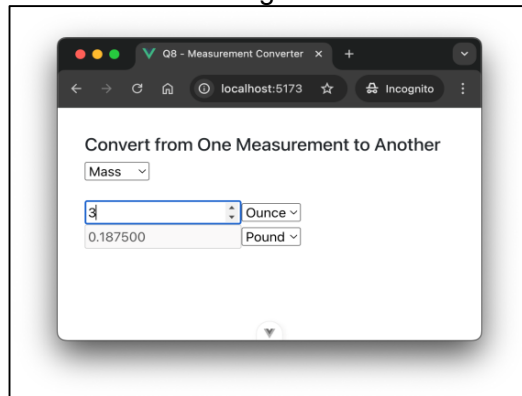


Figure q8j